Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Sistemas Operativos

Ano Letivo de 2025/2026

Grupo 31

Hugo Rauber
A104534

Tomás Machado
A104186

Tomás Ogando
A93174

May 10, 2026

SO

Índice

1. Introdução **1**

2. Arquitetura do Sistema **2**
2.1. Desenho da Solução 2
2.2. Gestão de Estado e Escalonamento 3
2.2.1. Políticas de Escalonamento 3

3. Comunicação Inter-Processo (IPC) e Sincronização **4**
3.1. Protocolo de Comunicação e Máquina de Estados 4
3.2. Persistência de Dados e Atomicidade 4

4. Implementação do Controller **5**
4.1. Ciclo de Execução Principal 5
4.1.1. Gestão de Processos e Monitorização 5
4.2. Gestão de Concorrência e Paralelismo 5

5. Implementação do Runner **7**
5.1. Ciclo de Vida e Estados 7
5.1.1. Execução de Pipelines e Redirecionamentos 7
5.1.2. Gestão de Memória e Buffers 7

6. Testes **8**
6.1. Metodologia de Testes 8
6.2. Avaliação de Desempenho e Resultados Experimentais 8
6.2.1. Análise do Impacto do Paralelismo (Mecanismo de Concorrência) 8
6.2.2. Comportamento Cronológico das Políticas de Escalonamento 8
6.2.2.1. Análise Crítica dos Resultados 9

7. Conclusão **10**

8. Bibliografia **11**

1. Introdução

No âmbito da unidade curricular de Sistemas Operativos, o presente projeto foca-se no desenho e implementação de um sistema robusto de escalonamento e gestão de tarefas assíncrono, assente numa arquitetura cliente-servidor de baixo nível em ambiente Unix. O núcleo do sistema é composto por duas entidades distintas e coordenadas: o `Controller` (servidor central) e o `Runner` (programa cliente).

O principal desafio deste desenvolvimento reside na necessidade de conceber um ecossistema que contorne a natureza tipicamente bloqueante da execução de comandos e pipelines da Shell. Para tal, a solução proposta adota uma arquitetura multi-nível baseada em hierarquias de processos independentes. Esta abordagem garante que o servidor central permaneça permanentemente disponível para auditar o sistema ou aceitar novos pedidos, delegando a monitorização ativa e o redirecionamento de fluxos de dados a processos secundários de curta duração.

Adicionalmente, o projeto aborda problemas clássicos da gestão de recursos partilhados num sistema operativo, tais como:

- **Controlo de Concorrência:** Limitação estrita do paralelismo de comandos ativos (`parallel-commands`) para mitigar a exaustão de CPU.
- **Comunicação Inter-Processo (IPC):** Utilização combinada de *Named Pipes* (FIFOs) públicos e privados para a troca síncrona de estruturas de controlo, e *pipes* anónimos para o encadeamento de pipelines complexos.
- **Justiça no Escalonamento (Fairness):** Implementação de políticas de ordenação (como FCFS e Round-Robin por utilizador) para impedir a inanição (**starvation**) de recursos do servidor.
- **Resiliência e Persistência Atómica:** Mecanismos de tolerância a falhas na manipulação de memória (buffers de 4096 bytes) e escrita centralizada e imutável num histórico de auditoria em formato CSV.

Este relatório descreve as decisões de desenho arquitetural, a mecânica das chamadas de sistema aplicadas, a metodologia de testes de carga implementada e, por fim, a análise analítica dos resultados experimentais obtidos.

2. Arquitetura do Sistema

2.1. Desenho da Solução

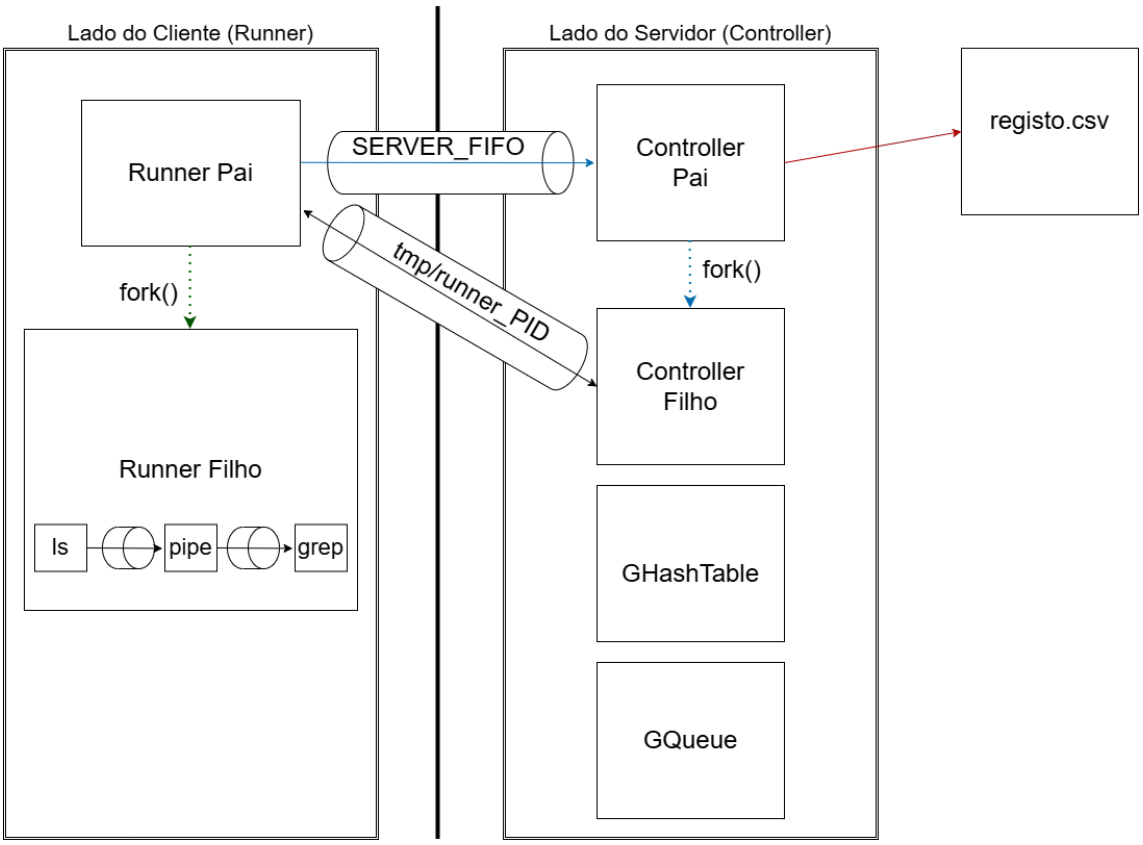

Figura 1: **Arquitetura do Sistema**

A arquitetura do sistema baseia-se num modelo *cliente-servidor assíncrono*, estruturado em quatro níveis de processos para garantir que o servidor nunca bloqueie durante a execução de comandos longos. O fluxo operacional segue uma sequência rigorosa de eventos:

- **Submissão e Delegação:** O `Runner Pai` inicia o protocolo enviando uma estrutura de dados para o *SERVER_FIFO*. O `Controller Pai`, ao ler o pedido, delega a gestão dessa tarefa específica a um `Controller Filho` (criado via *fork*), mantendo-se livre para aceitar novos pedidos ou consultas de estado.
- **Autorização e Execução:** O `Controller Filho` envia um sinal de autorização através de um *FIFO privado (/tmp/runner_PID)*. Ao receber este sinal, o `Runner Pai` cria o `Runner Filho`, que é responsável por configurar os pipes anónimos e redirecionamentos antes de se transformar no comando real através da chamada `execvp`.
- **Monitorização Ativa:** Enquanto o comando é executado, o `Runner Pai` permanece em espera ativa utilizando a função `wait`. Esta estratégia é fundamental, pois permite que o `Pai` retome o controlo assim que o `processo filho` termina, independentemente de qual seja o comando executado.

- **Finalização e Escrita de Dados:** Após a morte do `Runner Filho`, o `Runner Pai` envia a confirmação final (Tipo 4) ao `Controller Filho`. Este ciclo encerra-se quando o `Controller Pai` deteta o término do seu `processo filho`, procedendo ao cálculo da duração real da tarefa e à escrita atómica no ficheiro `registo.csv`, garantindo a integridade dos dados persistidos.

2.2. Gestão de Estado e Escalonamento

Para garantir uma gestão eficiente e de alto desempenho dos comandos, o `Controller` utiliza estruturas de dados avançadas da biblioteca `GLib`:

- **GHashTable (Comandos Ativos):** Utilizada para armazenar todos os processos que estão em execução num dado momento. O uso de uma tabela de dispersão permite que o `Controller Pai` aceda, insira ou remova informações sobre qualquer comando com complexidade temporal $O(1)$. Isto é crucial para que o servidor responda instantaneamente a pedidos de consulta de estado (`-c`).
- **GQueue (Fila de Espera):** Sempre que o limite de `parallel-commands` é atingido, os novos pedidos são inseridos em buffers de espera. A `GQueue` permite uma gestão eficiente de entrada e saída de processos, garantindo que nenhum pedido seja perdido enquanto o sistema está sob carga máxima.

2.2.1. Políticas de Escalonamento

O sistema foi desenhado para suportar diferentes algoritmos de ordenação, selecionáveis no arranque do servidor:

- **First-Come, First-Served (FCFS):** Implementada como a política base (identificador 0), utiliza uma `GQueue global` onde os pedidos são processados estritamente pela sua ordem de chegada ao *SERVER_FIFO*. É uma abordagem de baixa sobrecarga (overhead), mas que pode permitir o bloqueio do sistema por um único utilizador com grande volume de pedidos.
- **Round-Robin por Utilizador (Fairness):** Esta política (identificador 1) organiza os comandos em múltiplas filas indexadas por `user_id` numa `GHashTable` secundária. O sistema alterna ciclicamente entre utilizadores ativos, impedindo que um único utilizador monopolize todos os slots de execução disponíveis e garantindo justiça na distribuição de recursos.

3. Comunicação Inter-Processo (IPC) e Sincronização

3.1. Protocolo de Comunicação e Máquina de Estados

A comunicação é estruturada através da *struct Operacoes*, cujo **buffer** de comando foi expandido para 4096 bytes (inicialmente 1024) de forma a garantir a integridade da memória e evitar falhas de `stack smashing` em pipelines complexos. Em estrito cumprimento do enunciado, definiu-se que o identificador do utilizador (`user_id`) é fornecido via argumento do `Runner`, enquanto o identificador único do comando (`command_id`) é gerado de forma garantidamente unívoca através do `PID (Process ID)` do respetivo processo `Runner Pai` obtido via chamada de sistema `getpid()`. O sistema opera através de cinco estados críticos que definem o ciclo de vida de cada pedido:

- **Tipo 1 (Novo):** Emitido pelo `Runner Pai` para submeter uma tarefa ao *SERVER_FIFO*.
- **Tipo 2 (Consulta):** Pedido de estado do sistema, resultando na listagem de comandos em execução e agendados.
- **Tipo 3 (Shutdown):** Ordem de encerramento controlado do servidor.
- **Tipo 4 (Terminado):** Enviado pelo `Runner Pai` após o `wait()` confirmar a morte do processo filho, sinalizando ao servidor a libertação de recursos.
- **Tipo 5 (Handshake):** Enviado pelo `Runner Filho` imediatamente antes do `execvp`. Este estado é o marco temporal que permite ao `Controller` distinguir o tempo de permanência em fila do tempo de execução real.

3.2. Persistência de Dados e Atomicidade

A persistência é realizada no ficheiro `registo.csv`. Para garantir a integridade dos dados e evitar *condições de corrida (race conditions)*, implementaram-se as seguintes estratégias:

- **Escrita Centralizada:** Apenas o `Controller Pai` possui permissão de escrita no ficheiro, centralizando as atualizações após detetar a terminação de um `Controller Filho`.
- **Cabeçalho Atómico:** Utiliza-se a função `lseek()` para verificar o tamanho do ficheiro no momento da abertura. O cabeçalho CSV é injetado apenas se o deslocamento for zero, garantindo que a estrutura do ficheiro se mantém correta mesmo após múltiplos reinícios do servidor.
- **Chamadas de Sistema de Baixo Nível:** O uso de write() em vez de fprintf() assegura que os dados são enviados diretamente para o descritor de ficheiro do sistema operativo, minimizando problemas de *buffering*.

4. Implementação do Controller

4.1. Ciclo de Execução Principal

O `Controller` foi implementado como um servidor iterativo que utiliza um ciclo infinito para monitorizar o *SERVER_FIFO*. A lógica de processamento baseia-se na leitura de estruturas `Operacoes`, onde o comportamento do servidor é determinado pelo campo `tipo_operacao`:

- **Pedidos de Execução (Tipo 1):** O servidor valida o `user_id` e verifica a disponibilidade de slots de processamento. Se o limite for atingido, o pedido é inserido na fila; caso contrário, é despachado imediatamente.
- **Sincronização de Término (Tipo 4):** Ao receber este sinal, o servidor identifica o comando concluído, liberta o slot ocupado e verifica a existência de comandos pendentes na fila para promoção imediata.
- **Gestão de Handshake (Tipo 5):** O servidor regista o momento exato em que o `Runner Filho` inicia a execução, permitindo o cálculo posterior da duração real do processo.

4.1.1. Gestão de Processos e Monitorização

A implementação utiliza uma estratégia de delegação de tarefas para manter a disponibilidade do servidor:

- **Delegação via Fork:** Para cada comando autorizado, o `Controller Pai` cria um `processo filho (Controller Filho)`. Este processo é responsável por abrir o *FIFO* privado do cliente e gerir a comunicação de estado específica dessa tarefa.
- **Recolha de Processos:** O servidor implementa um mecanismo de monitorização (via wait ou sinais) para detetar o fim dos processos filhos. Esta etapa é crucial para que o `Controller Pai` possa atualizar as estruturas de dados e proceder à escrita dos tempos de execução no ficheiro de registo.
- **Encerramento Controlado:** No estado de `Shutdown (Tipo 3)`, o servidor deixa de aceitar novos pedidos e aguarda a conclusão de todos os processos atualmente em execução e os processos em fila antes de fechar os descritores de ficheiro e eliminar o *SERVER_FIFO*.

4.2. Gestão de Concorrência e Paralelismo

O controlo do paralelismo é o núcleo da eficiência do `Controller`. O limite de execução, definido pelo argumento `parallel-commands`, dita o número máximo de processos que podem coexistir no estado de `Executing`.

- **Controlo Ativo:** O sistema utiliza uma variável de contagem global que é incrementada ao autorizar um comando e decrementada ao receber a confirmação de término (Tipo 4).

- **Gestão de Excedentes:** Quando o limite é atingido, os novos pedidos (Tipo 1) são automaticamente movidos para uma *GQueue*, permanecendo no estado `Scheduled` até que uma vaga seja libertada.

5. Implementação do Runner

5.1. Ciclo de Vida e Estados

O Runner foi implementado para funcionar como uma interface síncrona que transita por estados bem definidos para garantir a coordenação com o servidor:

- **Handshake Inicial:** Após submeter o pedido, o `Runner Pai` bloqueia a sua execução num read() dirigido ao seu *FIFO privado (/tmp/runner_PID)*. Esta paragem é interrompida apenas quando o `Controller Filho` envia o sinal de autorização.
- **Supervisão (Wait):** Após criar o processo filho para execução, o `Runner Pai` utiliza waitpid() para monitorizar o estado do comando. Esta estratégia permite que o `Runner` recupere o controlo assim que o binário externo termina, possibilitando o envio do sinal de finalização (Tipo 4) ao servidor.

5.1.1. Execução de Pipelines e Redirecionamentos

A lógica de execução reside na função `execute_pipeline`, que processa a string do comando de forma iterativa:

- **Decomposição do Comando:** A string é fragmentada com base no operador `|`. Para cada segmento, é criado um novo processo. A comunicação entre estes processos é estabelecida através da chamada ao sistema `pipe()`, ligando o `STDOUT` de um comando ao `STDIN` do seguinte.
- **Manipulação de Descritores:** Utilizou-se a função `dup2()` para redirecionar as entradas e saídas padrão. Isto permite que o sistema suporte não só encadeamentos de comandos, mas também redirecionamentos para ficheiros `(>, <, 2>)`, garantindo que o fluxo de dados segue a lógica definida pelo utilizador na `Shell`.
- **Isolamento via Execvp:** Cada comando individual no pipeline sofre uma mutação via `execvp()`. Ao ser executado num processo neto, qualquer falha ou terminação do comando externo não afeta a estabilidade do `Runner Pai` nem do `Controller`.

5.1.2. Gestão de Memória e Buffers

Para suportar comandos de elevada complexidade e múltiplos argumentos, o sistema de leitura de argumentos foi redimensionado:

- **Redimensionamento de Buffers:** Expandiram-se os buffers internos para 4096 bytes (inicialmente 1024). Esta alteração foi fundamental para eliminar erros de `stack smashing` detetados durante os testes de carga com argumentos extensos, garantindo que a memória do processo permanece íntegra durante o parsing do pipeline.

6. Testes

6.1. Metodologia de Testes

A validação do sistema foi realizada através de uma bateria de testes automatizados, utilizando o script bash *teste_stress.sh*. A metodologia focou-se em três eixos principais:

- **Testes de Unidade e Integração:** Verificação individual de comandos simples e, posteriormente, de pipelines complexos com redirecionamentos, garantindo que o `Runner` e o `Controller` comunicam corretamente sem perda de mensagens.
- **Carga Concorrente:** Submissão massiva de pedidos para testar o comportamento da *GQueue* e a transição de estados de `Scheduled` para `Executing` assim que os slots de paralelismo ficam disponíveis.
- **Resiliência e Casos de Erro:** Injeção de entradas inválidas, comandos inexistentes e argumentos excessivamente longos para validar a robustez dos buffers de 4096 bytes e a capacidade do servidor em ignorar lixo nos *FIFOs* sem interromper o serviço.

6.2. Avaliação de Desempenho e Resultados Experimentais

A eficiência e o comportamento do sistema foram validados quantitativamente através de medições temporais obtidas diretamente nos logs de execução do script de stress e nos registos persistidos no ficheiro `registo.csv`.

6.2.1. Análise do Impacto do Paralelismo (Mecanismo de Concorrência)

Para medir o impacto real do argumento `parallel-commands` no escoamento de tarefas, utilizou-se uma carga de trabalho composta por uma rajada concorrente de 4 processos independentes, cada um executando uma instrução de suspensão controlada de 2 segundos (`sleep 2`). O sistema foi avaliado sob dois extremos de concorrência:

- **Cenário Sequencial (Limite = 1):** Configurado com concorrência unitária (Teste 2). Como o multiplicador de paralelismo obriga à execução isolada, as tarefas acumulam-se sequencialmente na fila de espera. O tempo total acumulado observado foi de **8 segundos** (tempo linear igual à soma individual de cada processo: $2s \times 4$).
- **Cenário de Concorrência Máxima (Limite = 4):** Configurado para permitir a coexistência total da carga (Teste 3). O `Controller` despachou os 4 comandos em simultâneo para o estado `Executing`. O tempo total observado para a conclusão de toda a bateria caiu para **2 segundos**, provando uma concorrência ideal onde o tempo total do lote passa a ser ditado unicamente pela duração da tarefa mais longa.

6.2.2. Comportamento Cronológico das Políticas de Escalonamento

O Teste 4 avaliou o impacto prático do escalonamento **Round-Robin por utilizador** na mitigação da inanição (**starvation**) de recursos. Configurou-se o `Controller` com `parallel-`

`commands = 2`, injetando um lote concorrente de 5 comandos de 5 segundos cada, distribuídos por 3 utilizadores.

A tabela seguinte apresenta os tempos totais de rotação (**Turnaround Time**) registados de forma atómica no ficheiro `registo.csv`:

User ID	Command ID	Duração Total (ms)
1	2412	5004.03
2	2413	5021.22
3	2414	10035.47
1	2415	10040.18
2	2416	16297.30

6.2.2.1. Análise Crítica dos Resultados

A progressão dos tempos em patamares incrementais (5s, 10s e 16s) valida empiricamente o comportamento do algoritmo de **Fair Scheduling**:

1. **Janela Inicial (5s):** Os comandos `2412` (User 1) e `2413` (User 2) ocuparam imediatamente as duas vagas de paralelismo, terminando sem tempo de retenção em fila.
2. **Segunda Janela (10s):** Com a libertação de vagas, evitou-se o cenário de uma política estritamente cronológica **(FCFS)**, onde o `User 2` manteria prioridade absoluta por ordem de chegada. Em vez disso, o escalonador **Round-Robin** preteriu o segundo pedido acumulado do utilizador monopolizador, promovendo o `User 3` (comando `2414`) e intercalando o pedido `2415` do `User 1`. Ambos acumularam 5s de espera com 5s de execução útil.
3. **Penalização por Sobrecarga (16s):** O comando `2416` (User 2) aguardou uma rotação circular completa, acumulando duas janelas de contenção em fila (10s) antes de ser autorizado a executar.

Em suma, enquanto o algoritmo **FCFS** penalizaria utilizadores posteriores devido ao efeito de **Convoy**, esta alternância baseada na identidade do utilizador (gerida por `GHashTables` de filas) provou ser eficaz no isolamento de submissões em rajada massiva, garantindo acesso equitativo aos recursos e erradicando cenários de inanição (**starvation**), conforme proposto no enunciado.

7. Conclusão

O desenvolvimento deste sistema de gestão de tarefas permitiu a aplicação prática de conceitos fundamentais de Sistemas Operativos, resultando numa solução robusta, escalável e resiliente. A arquitetura implementada, baseada numa hierarquia de quatro níveis de processos, provou ser eficaz no isolamento de falhas e na manutenção da disponibilidade do servidor, mesmo sob condições de carga elevada.

A utilização de mecanismos de comunicação inter-processo, como os `Named Pipes (FIFOs)` e Pipes anónimos, garantiu uma troca de dados fluida entre o `Controller` e o `Runner`. O rigoroso protocolo de comunicação estabelecido, reforçado pelo uso do `Handshake (Tipo 5)`, foi determinante para separar com precisão o tempo de espera em fila do tempo de execução real, oferecendo métricas de performance fiáveis.

Ao nível do servidor, a integração da biblioteca *GLib* ofereceu ao `Controller` uma gestão de estado eficiente através de *GHashTables* e *GQueues*, permitindo um escalonamento justo (`Round-Robin`) que evita a inanição (`starvation`) dos utilizadores. Por outro lado, o `Runner` demonstrou versatilidade ao suportar operadores complexos da `Shell`, como encadeamentos e redirecionamentos, mantendo a integridade da memória através de `buffers` redimensionados para mitigar falhas de `stack smashing`.

Os testes experimentais validaram as decisões de desenho, evidenciando uma redução de 75 % no tempo de processamento ao transitar para um modelo de execução paralelo. Em suma, o projeto cumpre todos os requisitos propostos, oferecendo uma ferramenta capaz de gerir múltiplos processos de forma assíncrona, garantindo a persistência atómica dos dados e a estabilidade global do sistema.

8. Bibliografia

[1] GNOME Project, «GLib Reference Manual». [Em linha]. Disponível em: https://docs.gtk.org/glib/

[2] Michael Kerrisk, «Linux Manual Pages». [Em linha]. Disponível em: https://man7.org/linux/man-pages/

[3] «Diagrams.net (draw.io)». [Em linha]. Disponível em: https://app.diagrams.net/

[4] The IEEE and The Open Group, «The Open Group Base Specifications Issue 7 (IEEE Std 1003.1)».